# Canadian Freelancer Invoicing Tool

Full Product Spec — Next.js Only Stack — Ready for Claude Code / Cursor

## What Are We Building?

A Next.js web app where Canadian freelancers log in, build invoices, and get correct Canadian tax logic automatically. One codebase, one Vercel deployment, no backend server. The whole product lives in Next.js API routes + Supabase. That's it.

## Why Next.js Only (No Django)

- One repo, one codebase, one Vercel deploy
- API routes handle all backend logic — tax calc, PDF gen, emails, webhooks
- Supabase handles auth + database — no extra server needed
- Faster to build, easier to maintain solo
- You already know this stack from your other projects

## Tech Stack

| Layer | Technology | What It Does |
|---|---|---|
| Framework | Next.js 14 (App Router) | Frontend + all API routes in one project |
| Database | Supabase (PostgreSQL) | Stores users, invoices, clients |
| Auth | Supabase Auth | Email + Google login, built in |
| State | Zustand | Client-side state management |
| Payments | Lemon Squeezy | Subscriptions, handles all tax compliance |
| PDF | @react-pdf/renderer | Generate invoice PDFs in Next.js |
| Email | Resend | Send invoices to clients by email |
| Styling | Tailwind CSS | Fast clean UI |
| Hosting | Vercel | One click deploy, free tier works |
| Types | TypeScript | Avoid silly bugs, better DX |

## Project Folder Structure

Paste this into Claude Code to set up the project skeleton:

```
/app
/app/(auth)/login/page.tsx — Login page
/app/(auth)/signup/page.tsx — Signup page
/app/(dashboard)/dashboard/page.tsx — Invoice list
/app/(dashboard)/invoices/new/page.tsx — Invoice builder
```

```
/app/(dashboard)/invoices/[id]/page.tsx — Invoice detail
/app/(dashboard)/clients/page.tsx — Client manager
/app/(dashboard)/settings/page.tsx — Settings
/app/api
/app/api/invoices/route.ts — GET list, POST create
/app/api/invoices/[id]/route.ts — GET, PUT, DELETE
/app/api/invoices/[id]/pdf/route.ts — Generate PDF
/app/api/invoices/[id]/send/route.ts — Send email
/app/api/clients/route.ts — GET list, POST create
/app/api/clients/[id]/route.ts — GET, PUT, DELETE
/app/api/tax/calculate/route.ts — Tax calculation logic
/app/api/webhooks/lemonsqueezy/route.ts — Payment webhooks
/app/api/profile/route.ts — GET/PUT business profile
/lib
/lib/supabase.ts — Supabase client
/lib/tax.ts — All Canadian tax logic
/lib/pdf.ts — PDF generation helpers
/lib/email.ts — Resend email helpers
/lib/lemonsqueezy.ts — Payment helpers
/store
/store/invoiceStore.ts — Zustand invoice state
/store/clientStore.ts — Zustand client state
/types
/types/invoice.ts — TypeScript types
/types/client.ts
/types/tax.ts
```

# Supabase Database Tables

Run these as SQL in Supabase dashboard or paste into Claude Code to generate migration:

## profiles

```
id uuid references auth.users primary key
business_name text
full_name text
province text -- e.g. 'NS', 'ON', 'BC'
gst_hst_number text
qst_number text -- Quebec only
is_gst_registered boolean default false
default_currency text default 'CAD'
```

```
logo_url text
payment_instructions text
invoice_number_format text default 'INV-{number}'
subscription_status text default 'trialing' -- trialing/active/cancelled
lemon_squeezy_customer_id text
created_at timestamptz default now()
```

## clients

```
id uuid default gen_random_uuid() primary key
user_id uuid references profiles(id)
name text not null
email text
country text not null -- 'CA' or 'US' or other
province text -- only if country = 'CA'
created_at timestamptz default now()
```

## invoices

```
id uuid default gen_random_uuid() primary key
user_id uuid references profiles(id)
client_id uuid references clients(id)
invoice_number text not null
issue_date date not null
due_date date not null
currency text default 'CAD'
status text default 'draft' -- draft/sent/paid/overdue
notes text
subtotal numeric(10,2)
tax_total numeric(10,2)
total numeric(10,2)
created_at timestamptz default now()
```

## invoice_line_items

```
id uuid default gen_random_uuid() primary key
invoice_id uuid references invoices(id) on delete cascade
description text not null
quantity numeric(10,2) default 1
rate numeric(10,2) not null
amount numeric(10,2) not null -- quantity * rate
sort_order int default 0
```

### invoice_taxes

```
id uuid default gen_random_uuid() primary key
invoice_id uuid references invoices(id) on delete cascade
tax_name text not null -- e.g. 'HST', 'GST', 'PST', 'QST'
tax_rate numeric(5,4) not null -- e.g. 0.15 for 15%
tax_amount numeric(10,2) not null
note text -- e.g. zero-rated note for international clients
```

## The Tax Logic — lib/tax.ts

This is the entire value proposition of the product. Every edge case matters. Write unit tests for every province and the international case.

### Rule 1 — International client (not Canadian):

- Return 0% tax
- Add note: 'Zero-rated export supply — GST/HST not applicable (ETA s.123)'
- Still show the zero on the invoice so client knows you thought about it

### Rule 2 — Seller NOT GST/HST registered:

- Return no tax lines at all
- Add note: 'Small supplier — not registered for GST/HST'

### Rule 3 — Canadian client, seller IS registered:

| Client Province | Tax Type | Rate(s) | Lines on Invoice |
|---|---|---|---|
| Ontario | HST | 13% | One line: HST 13% |
| Nova Scotia | HST | 15% | One line: HST 15% |
| New Brunswick | HST | 15% | One line: HST 15% |
| PEI | HST | 15% | One line: HST 15% |
| Newfoundland & Labrador | HST | 15% | One line: HST 15% |
| British Columbia | GST + PST | 5% + 7% | Two lines: GST 5%, PST 7% |
| Alberta | GST only | 5% | One line: GST 5% |
| Saskatchewan | GST + PST | 5% + 6% | Two lines: GST 5%, PST 6% |
| Manitoba | GST + RST | 5% + 7% | Two lines: GST 5%, RST 7% |
| Quebec | GST + QST | 5% + 9.975% | Two lines: GST 5%, QST 9.975% |
| Territories (YT/NT/NU) | GST only | 5% | One line: GST 5% |

### TypeScript function signature to implement:

```
calculateTax(sellerProvince: string, clientCountry: string, clientProvince: string | null,
subtotal: number, isGstRegistered: boolean): TaxLine[]
```

```
type TaxLine = { name: string; rate: number; amount: number; note?: string }
```

# Next.js API Routes

| Method | Route | What It Does |
|---|---|---|
| GET | /api/invoices | Return all invoices for logged-in user |
| POST | /api/invoices | Create new invoice + line items + taxes |
| GET | /api/invoices/[id] | Get single invoice with all related data |
| PUT | /api/invoices/[id] | Update invoice |
| DELETE | /api/invoices/[id] | Delete invoice |
| GET | /api/invoices/[id]/pdf | Generate + return PDF as blob |
| POST | /api/invoices/[id]/send | Email PDF to client via Resend |
| POST | /api/invoices/[id]/mark-paid | Update status to paid |
| GET | /api/clients | List all clients |
| POST | /api/clients | Create client |
| PUT | /api/clients/[id] | Update client |
| DELETE | /api/clients/[id] | Delete client |
| POST | /api/tax/calculate | Run tax logic, return tax lines |
| GET | /api/profile | Get business profile |
| PUT | /api/profile | Update business profile |
| POST | /api/webhooks/lemonsqueezy | Handle subscription events |

# The 7 Screens

## 1. Onboarding — /onboarding

- Runs once after signup
- Fields: Full name, Business name, Province (dropdown), GST/HST number (optional), default currency
- Toggle: 'I am registered for GST/HST' — this changes all tax behaviour
- Logo upload → stored in Supabase Storage
- On save → redirect to /dashboard

## 2. Dashboard — /dashboard

- Table: Invoice #, Client, Amount (with currency), Due Date, Status badge
- Status colours: grey=Draft, blue=Sent, green=Paid, red=Overdue
- Stats row at top: Total Outstanding | Paid This Month | Number of Overdue
- New Invoice button → /invoices/new
- Click row → /invoices/[id]
- Empty state with illustration + 'Create your first invoice' CTA

## 3. Invoice Builder — /invoices/new

- Client picker: dropdown of saved clients OR 'Add new client' inline

- Invoice #: auto-generated, editable
- Issue date (today default), Due date (today +30 days default)
- Currency toggle: CAD / USD
- Line items: add/remove rows, each has Description + Qty + Rate = Amount
- Tax section: auto-calculates based on client country + province + your settings
- Shows each tax line separately (e.g. GST $25.00, PST $35.00)
- For international clients: shows zero-rated note automatically
- Notes / payment instructions field (prefilled from Settings)
- Bottom actions: Save Draft | Preview PDF | Send to Client

## 4. Invoice Detail — /invoices/[id]

- Read-only view of the invoice
- Status badge + last updated
- Action buttons: Edit (if draft) | Download PDF | Send Email | Mark as Paid | Duplicate | Delete
- Timeline/activity: Created, Sent on [date], Paid on [date]

## 5. Client Manager — /clients

- Table of all saved clients: Name, Email, Country, Last Invoiced, Total Billed
- Add client button → modal with: Name, Email, Country, Province (if Canada)
- Click client → edit modal
- Delete with confirmation

## 6. Settings — /settings

- Business profile: name, province, GST/HST number, QST number (Quebec)
- GST/HST registered toggle
- Logo upload
- Default payment instructions
- Invoice number format
- Subscription section: current plan, next billing date, Manage Subscription button (Lemon Squeezy portal)

## 7. PDF Invoice (generated, not a page)

- Your logo top left, Invoice title top right
- Your business info + GST/HST number
- Client info block
- Invoice #, Issue Date, Due Date
- Line items table with subtotal
- Tax lines with rates clearly shown
- Zero-rated note if international client
- Total in large text

• Payment instructions at bottom

• Professional, clean — think Stripe invoices as reference

# Lemon Squeezy Setup

• Sign up at lemonsqueezy.com — takes 10 minutes

• Create Store → Create Product: 'Canadian Invoice Tool'

• Create 2 variants: Monthly ($19 CAD) and Annual ($149 CAD)

• Get your API key + store ID from dashboard

• Add to .env.local: LEMONSQUEEZY_API_KEY, LEMONSQUEEZY_STORE_ID, LEMONSQUEEZY_WEBHOOK_SECRET

• Checkout: redirect user to Lemon Squeezy hosted checkout URL with their email pre-filled

• Webhook at /api/webhooks/lemonsqueezy — listen for:

```
subscription_created → set profiles.subscription_status = 'active'
subscription_cancelled → set profiles.subscription_status = 'cancelled'
subscription_expired → set profiles.subscription_status = 'expired'
```

• Gate invoice creation: check subscription_status === 'active' in API routes

• Free trial: allow 3 invoices without subscription, then show paywall

# Environment Variables (.env.local)

```
NEXT_PUBLIC_SUPABASE_URL=your_supabase_url
NEXT_PUBLIC_SUPABASE_ANON_KEY=your_supabase_anon_key
SUPABASE_SERVICE_ROLE_KEY=your_service_role_key
LEMONSQUEEZY_API_KEY=your_api_key
LEMONSQUEEZY_STORE_ID=your_store_id
LEMONSQUEEZY_WEBHOOK_SECRET=your_webhook_secret
RESEND_API_KEY=your_resend_api_key
NEXT_PUBLIC_APP_URL=https://yourdomain.com
```

# 3-Week Build Order

## Week 1 — Foundation

• Day 1: npx create-next-app, Supabase setup, auth (login/signup pages), profiles table

• Day 2: Supabase tables — clients, invoices, invoice_line_items, invoice_taxes

• Day 3: lib/tax.ts — full Canadian tax logic with TypeScript types + unit tests

• Day 4: /api/invoices routes + /api/clients routes + /api/tax/calculate

• Day 5: Zustand stores for invoice and client state

## Week 2 — UI

• Day 1: Dashboard page — invoice list table, stats row, empty state

• Day 2: Invoice builder — line items, client picker, currency toggle

- Day 3: Tax section UI in invoice builder — auto-updates as you change client
- Day 4: Client manager page + Settings page
- Day 5: PDF generation with @react-pdf/renderer — professional invoice layout

### Week 3 — Ship It

- Day 1: Resend email integration — send PDF as attachment to client
- Day 2: Lemon Squeezy checkout + webhook handler + subscription gating
- Day 3: Onboarding flow + invoice detail page + mark as paid
- Day 4: Deploy to Vercel, connect custom domain, test everything end to end
- Day 5: Landing page + first 5 SEO blog posts published

## Prompts to Paste Into Claude Code / Cursor

Use these one at a time. Complete each before moving to the next.

### 1. Project setup:

```
Create a Next.js 14 App Router project with TypeScript, Tailwind CSS, and Supabase auth. Set up the folder structure with app/(auth)/login, app/(auth)/signup, app/(dashboard)/dashboard, app/(dashboard)/invoices/new, app/(dashboard)/invoices/[id], app/(dashboard)/clients, app/(dashboard)/settings. Add Zustand. Create a Supabase client in lib/supabase.ts with server and client helpers.
```

### 2. Supabase tables:

```
Write the SQL to create these Supabase tables: profiles (id, business_name, full_name, province, gst_hst_number, qst_number, is_gst_registered, default_currency, logo_url, payment_instructions, invoice_number_format, subscription_status, lemon_squeezy_customer_id), clients (id, user_id, name, email, country, province), invoices (id, user_id, client_id, invoice_number, issue_date, due_date, currency, status, notes, subtotal, tax_total, total), invoice_line_items (id, invoice_id, description, quantity, rate, amount, sort_order), invoice_taxes (id, invoice_id, tax_name, tax_rate, tax_amount, note). Add RLS policies so users can only see their own data.
```

### 3. Tax logic:

```
Create lib/tax.ts with a TypeScript function calculateTax(sellerProvince: string, clientCountry: string, clientProvince: string | null, subtotal: number, isGstRegistered: boolean): TaxLine[]. Handle: 1) international clients = zero-rated with note, 2) seller not registered = no tax with note, 3) Canadian clients with correct rates per province: ON=HST 13%, NS/NB/PEI/NL=HST 15%, BC=GST 5%+PST 7%, AB=GST 5%, SK=GST 5%+PST 6%, MB=GST 5%+RST 7%, QC=GST 5%+QST 9.975%, territories=GST 5%. Write Jest unit tests for every case.
```

### 4. Invoice API routes:

```
Create Next.js API routes for invoices: GET /api/invoices (list user invoices), POST /api/invoices (create invoice with line items and taxes in a transaction), GET /api/invoices/[id] (get invoice with line items, taxes, and client), PUT /api/invoices/[id] (update), DELETE /api/invoices/[id] (delete), POST /api/invoices/[id]/mark-paid (update
```

```
status). Use Supabase service role key server-side. Check subscription_status on create — allow max 3 invoices if not active subscriber.
```

### 5. Invoice builder UI:

```
Create the invoice builder page at app/(dashboard)/invoices/new/page.tsx. Include: client picker dropdown (fetches from /api/clients) with Add New Client option, invoice number field, issue date + due date pickers, currency toggle CAD/USD, dynamic line items table (add/remove rows, auto-calculate amount = qty * rate), tax section that calls /api/tax/calculate when client or line items change and displays each tax line, notes textarea, total summary, Save Draft and Send to Client buttons. Use Tailwind CSS, Zustand for state.
```

### 6. PDF generation:

```
Create a PDF invoice component using @react-pdf/renderer. The PDF should have: logo top left, Invoice title + number top right, seller business info + GST/HST number, client info block, invoice dates, line items table with columns Description/Qty/Rate/Amount, subtotal row, individual tax lines with rates, zero-rated note if applicable, bold total, payment instructions at bottom. Create API route GET /api/invoices/[id]/pdf that fetches invoice data and returns the PDF as a blob response.
```

### 7. Lemon Squeezy:

```
Integrate Lemon Squeezy payments. Create lib/lemonsqueezy.ts with a function to generate a checkout URL for monthly ($19) and annual ($149) plans with the user's email pre-filled. Create POST /api/webhooks/lemonsqueezy route that verifies the webhook signature using LEMONSQUEEZY_WEBHOOK_SECRET, then handles subscription_created (set subscription_status=active), subscription_cancelled (set subscription_status=cancelled), subscription_expired (set subscription_status=expired) by updating the profiles table in Supabase.
```

## SEO Pages to Write on Launch Week

Publish these as blog posts under /blog. Target exact-match keywords Canadians are googling.

- Do I Charge HST to US Clients? (The Definitive Answer for Canadian Freelancers)
- Canadian Freelancer's Complete Guide to GST/HST
- How to Invoice International Clients from Canada — Zero-Rated Explained
- Free HST Calculator for Canadian Freelancers 2024
- Nova Scotia Freelancer Tax Guide — HST, Income Tax, and Invoicing
- Should I Register for GST/HST? The $30K Threshold Explained
- GST vs HST vs PST — What's the Difference? (Canadian Freelancer Guide)
- QST for Quebec Freelancers — Complete 2024 Guide
- How to Invoice US Clients from Canada Without Getting Taxes Wrong
- Canadian Freelance Invoice Template — Free Download with Correct Tax Logic

---

Built for Michelle — Halifax solo dev, 2026. Stack: Next.js 14 + Supabase + Zustand + Lemon Squeezy + Resend. One repo. One deploy. Ship it.